CHAPTER 5

Intuitionism and the Meaning of Proof

Chapter 4 ended with a puzzle. The formula $A \vee \neg A$, "every proposition is true or false", holds in every row of its truth table, yet the rules of natural deduction cannot derive it. And in Chapter 1, `#print axioms` showed that a proof by the `grind` tactic relied on something called `Classical.choice`, while a hand-written proof did not. Both observations point to the same fact: there are two respectable notions of proof, and the logic we have been using is the more demanding one.

That logic is called **intuitionistic** (or constructive). In it, to prove "A or B" you must say which, and to prove "there exists an x" you must produce one. This chapter explains where the idea comes from, what it costs (some classical laws fail), what it buys (proofs that carry information, which is why it is the logic of programming), and why classical logic is not wrong but sits inside it. We then build two semantics in which intuitionistic logic, unlike truth tables, is exactly right: Kripke models and Heyting algebras. With them we finally prove in Lean that excluded middle is not derivable in the system of Chapter 4.

In this chapter you will learn:

- the Brouwer–Heyting–Kolmogorov (BHK) interpretation, in which a proof of $A \vee B$ is a proof of one side *together with the information which*, and a proof of $A \to B$ is a method;
- which classical laws fail constructively (excluded middle, double-negation elimination, Peirce's law, one De Morgan law), and what you gain in exchange: the disjunction and existence properties;
- why classical logic is not wrong: the double-negation translation, and the constructive proof of $\neg\neg(A \vee \neg A)$;
- how negation as $A \to \bot$ looks in Rust, with empty types, `Infallible` and the never type;
- how Lean separates the two worlds: `Classical.em`, `Classical.byContradiction`, `Decidable`, and `#print axioms`;
- Kripke semantics, a two-world countermodel to excluded middle checked by `decide`, and an exhaustive countermodel search in Rust;
- Heyting algebras, built from scratch in Lean and then taken from Mathlib, and a first look at the categorical reading of logic.

5.1 Brouwer and the meaning of "true"

Intuitionism began in the early twentieth century with the Dutch mathematician L. E. J. Brouwer, as a philosophy of mathematics rather than a logic. Its central claim is that mathematical truth *is* provability: to assert A is to possess a construction, a mental proof, of A, not to report a fact about a realm that exists independently of what we can establish. Brouwer was suspicious of formal logic; the formal system was written down by his student Arend Heyting around 1930, and the logic it defines is exactly the logic of Chapter 4's rules.

The consequence for logic follows quickly. If asserting "A or B" means having a construction of A or having a construction of B, then asserting "$A \vee \neg A$ for every A" means claiming a method that, for every proposition, either proves it or refutes it. Nobody has such a method; for many open problems we have neither. So excluded middle cannot be asserted *in general*.

A classic example shows what a classical proof can leave out. *There exist irrational numbers a and b such that a^b is rational.* Proof: consider $\sqrt{2}^{\sqrt{2}}$. If it is rational, take $a = b = \sqrt{2}$. If not, take $a = \sqrt{2}^{\sqrt{2}}$ and $b = \sqrt{2}$, since then $a^b = \sqrt{2}^{\sqrt{2}\cdot\sqrt{2}} = \sqrt{2}^2 = 2$. The proof is classically impeccable, and at the end you still do not know which pair works. It used excluded middle on a question it never answered. An intuitionist does not say the proof is wrong; they say it proves something weaker than it claims: that the non-existence of such a pair is absurd.

> **NOTE**
>
> Intuitionism does not claim that excluded middle is *false*. As we will prove, $\neg\neg(A \vee \neg A)$ is intuitionistically provable: excluded middle can never be refuted. It is just not available as a general principle, because asserting a disjunction requires knowing which side holds.

5.2 The BHK interpretation

The **Brouwer–Heyting–Kolmogorov interpretation** says what counts as a proof of each kind of formula. Kolmogorov contributed a reading of propositions as *problems* and proofs as *solutions*, which is the same idea in a programmer's vocabulary. The table adds a column for Rust and one for Lean.

Formula	A proof is	Rust	Lean
$A \wedge B$	a proof of A and a proof of B	`(A, B)`	`⟨ha, hb⟩`
$A \vee B$	a tag saying which side, and a proof of it	`enum Either<A, B>`	`Or.inl ha`
$A \to B$	a method turning proofs of A into proofs of B	`fn(A) -> B`	`fun ha => ...`
$\top$	the trivial proof	`()`	`True.intro`
$\bot$	none exists	`enum Void {}`	(no constructor)
$\neg A$	a method turning proofs of A into proofs of $\bot$	`fn(A) -> Void`	`fun ha => ...`
$\forall x.\, A(x)$	a method giving, for each x, a proof of $A(x)$	generic `fn` (on types)	`fun x => ...`
$\exists x.\, A(x)$	a witness x and a proof of $A(x)$	`impl Trait` (on types)	`⟨x, h⟩`

Two observations. First, the BHK column *is* the type column: the explanation of the connectives that intuitionists gave in the 1930s is a description of data types, decades before Curry and Howard made the correspondence formal. That is why Chapter 10 can be short: the work is done here. (The Rust entries for the quantifiers are only approximations, since Rust cannot quantify over values; Chapter 10 discusses this.)

Second, the $\vee$ and $\exists$ rows are exactly where classical logic parts company. Both demand that a proof contain information: which side, which witness. A classical proof of $A \vee B$ may contain no such information, as the $\sqrt{2}$ example shows. Everything else in this chapter follows from taking those two rows seriously.

5.3 What fails, and what you gain

Excluded middle and its relatives

These classical laws have no intuitionistic proof:

Law	Formula
excluded middle	$A \vee \neg A$
double-negation elimination	$\neg\neg A \to A$
Peirce's law	$((A \to B) \to A) \to A$
one De Morgan law	$\neg(A \wedge B) \to \neg A \vee \neg B$
material implication	$(A \to B) \to \neg A \vee B$
converse contraposition	$(\neg B \to \neg A) \to (A \to B)$

Each has an easy classical proof. In Lean, classical reasoning is available through two library functions:

```
/-- info: Classical.em (p : Prop) : p ∨ ¬p -/
#guard_msgs in
#check Classical.em

/-- info: Classical.byContradiction {p : Prop} (h : ¬p → False) : p -/
#guard_msgs in
#check Classical.byContradiction

/-- info: axiom Classical.choice.{u} : {α : Sort u} → Nonempty α → α -/
#guard_msgs in
#print Classical.choice
```

`Classical.em` is excluded middle, and `Classical.byContradiction` is the rule RAA of Chapter 4: to prove `p`, show that `¬p` leads to `False`. Both are theorems, proved from the **axiom** `Classical.choice` (with help from two other axioms that we meet in a moment). Choice says that from the knowledge that a type is nonempty (`Nonempty α`, a proposition) you can obtain an element of it. That is precisely the step from "something exists" to "here it is" that BHK refuses. With these, the classical laws are short (the code is in `code/lean/Book/Ch05/Constructive.lean`):

```
/-- Double-negation elimination: proof by contradiction. -/
theorem dne (p : Prop) : ¬¬p → p := by
  intro hnnp
```

```
  apply Classical.byContradiction   -- goal: ¬p → False
  intro hnp
  exact hnnp hnp

/-- Peirce's law, by cases on excluded middle. -/
theorem peirce (p q : Prop) : ((p → q) → p) → p := by
  intro h
  cases Classical.em p with
  | inl hp => exact hp
  | inr hnp => exact h (fun hp => absurd hp hnp)

/-- The De Morgan law that fails constructively. `by_cases` splits on
`Classical.em`. -/
theorem not_and_or (p q : Prop) : ¬(p ∧ q) → ¬p ∨ ¬q := by
  intro h
  by_cases hp : p
  · right
    intro hq
    exact h ⟨hp, hq⟩
  · left
    exact hp
```

`dne` applies `byContradiction`, which turns the goal `p` into `¬p → False`. `peirce` splits on `Classical.em p`. `not_and_or` uses the tactic `by_cases`, which does the same split with a friendlier syntax. Now the tool that tells proofs apart, `#print axioms`, which you met in Chapter 1:

```
/-- info: 'Ch05.nn_em' does not depend on any axioms -/
#guard_msgs in
#print axioms nn_em

/-- info: 'Ch05.not_or_iff' does not depend on any axioms -/
#guard_msgs in
#print axioms not_or_iff

/-- info: 'Ch05.dne' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms dne

/-- info: 'Ch05.not_and_or' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms not_and_or
```

`nn_em` and `not_or_iff`, proved below, use no axioms at all: they are constructions in the BHK sense. `dne` and `not_and_or` depend on `Classical.choice` (and on `propext` and `Quot.sound`, two further axioms that Lean's library uses to derive excluded middle from choice; they are not classical by themselves). Whenever you want to know whether a Lean proof is constructive, this is the command to run.

All or nothing

Excluded middle, double-negation elimination and Peirce's law are not three separate principles. Over intuitionistic logic, each one, assumed for all propositions, implies the others. In Lean the implications are constructive proofs that take the principle as a hypothesis:

```
/-- If excluded middle holds for every proposition, so does double-negation
elimination... -/
theorem dne_of_em (em : ∀ p : Prop, p ∨ ¬p) : ∀ p : Prop, ¬¬p → p := by
  intro p hnnp
  cases em p with
  | inl hp => exact hp
  | inr hnp => exact absurd hnp hnnp
```

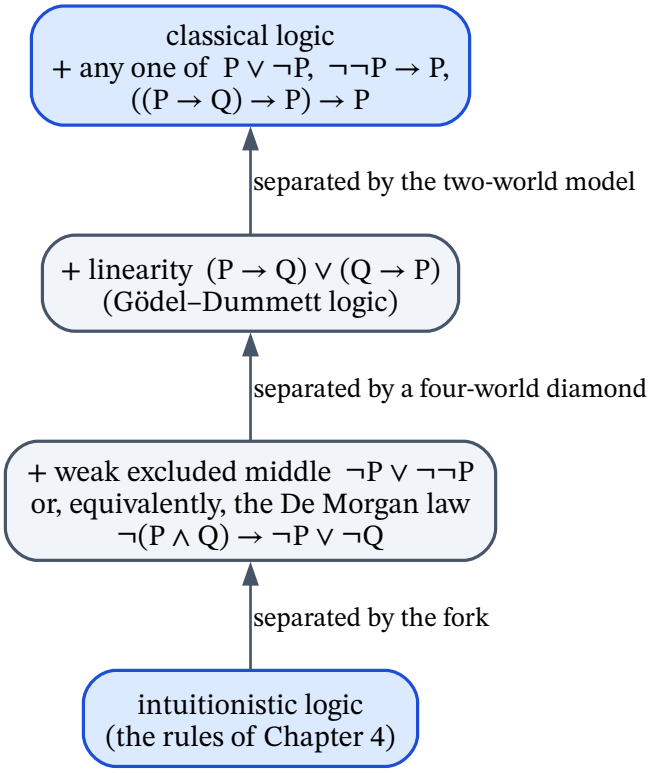
```
/-- ...and conversely: apply double-negation elimination to `nn_em`. -/
theorem em_of_dne (dne : ∀ p : Prop, ¬¬p → p) : ∀ p : Prop, p ∨ ¬p :=
  fun p => dne (p ∨ ¬p) (nn_em p)

/-- Peirce's law implies excluded middle: take `q := False` and the
proposition `p ∨ ¬p`. -/
theorem em_of_peirce (peirce : ∀ p q : Prop, ((p → q) → p) → p) :
    ∀ p : Prop, p ∨ ¬p := by
  intro p
  apply peirce (p ∨ ¬p) False   -- goal: (p ∨ ¬p → False) → p ∨ ¬p
  intro k                       -- k : ¬(p ∨ ¬p)
  exact (nn_em p k).elim

/-- info: 'Ch05.em_of_peirce' does not depend on any axioms -/
#guard_msgs in
#print axioms em_of_peirce
```

The first proof splits on the excluded-middle hypothesis. The second applies double-negation elimination to `nn_em`, the constructive proof of $\neg\neg(p \vee \neg p)$ that we study below. The third instantiates Peirce's law with `q := False`: its premise `(p ∨ ¬p → False) → p ∨ ¬p` is again discharged by `nn_em`. And `#print axioms` confirms that these implications are themselves constructive. Adding any one of the three principles to the rules of Chapter 4 gives full classical logic; there is no "slightly classical" logic made of just one of them.

The De Morgan law is different, and the difference is easy to miss. It is equivalent not to excluded middle but to the weaker **weak excluded middle**, $\neg A \vee \neg\neg A$. Between intuitionistic and classical logic there are in fact infinitely many **intermediate logics**, and the two weaker principles above define two of them:



A few logics between intuitionistic and classical logic. Each step adds a principle; the label says which Kripke model separates the two logics.

The models in the labels are the subject of the Kripke section below, where a Rust program finds them.

What survives

Much of everyday logic survives. All of these are constructive:

```
/-- Double-negation introduction: a proof of `p` refutes every refutation. -/
theorem dni (p : Prop) : p → ¬¬p :=
  fun hp hnp => hnp hp

/-- Triple negation collapses to single negation. -/
theorem tne (p : Prop) : ¬¬¬p → ¬p :=
  fun hnnnp hp => hnnnp (dni p hp)

/-- Contraposition, in the direction that holds. -/
theorem contra (p q : Prop) : (p → q) → (¬q → ¬p) :=
  fun hpq hnq hp => hnq (hpq hp)

/-- Two De Morgan laws hold in both directions or in one. -/
theorem not_or_iff (p q : Prop) : ¬(p ∨ q) ↔ ¬p ∧ ¬q :=
  ⟨fun h => ⟨fun hp => h (Or.inl hp), fun hq => h (Or.inr hq)⟩,
   fun ⟨hnp, hnq⟩ h => h.elim hnp hnq⟩

theorem not_and_of_or_not (p q : Prop) : ¬p ∨ ¬q → ¬(p ∧ q) :=
  fun h ⟨hp, hq⟩ => h.elim (fun hnp => hnp hp) (fun hnq => hnq hq)
```

Read them as programs. `dni` takes a proof `hp` and a refutation `hnp` and applies the second to the first. `tne` turns a refutation of $\neg\neg p$ into a refutation of p by feeding it `dni p hp`: triple negation collapses to single negation, although double negation does not collapse to nothing. `contra` is composition. In `not_or_iff`, a refutation of $p \vee q$ yields refutations of each side by composing with `Or.inl` and

`Or.inr`, and conversely a pair of refutations handles both cases of a disjunction. `not_and_of_or_not` is the direction of De Morgan's law that needs no information: if one side is refuted, the conjunction is.

Constructive	Classical only
$A \to \neg\neg A$	$\neg\neg A \to A$
$\neg\neg\neg A \leftrightarrow \neg A$	$A \vee \neg A$
$\neg(A \wedge \neg A)$	$\neg(A \wedge B) \to \neg A \vee \neg B$
$\neg\neg(A \vee \neg A)$	$(\neg B \to \neg A) \to (A \to B)$
$\neg(A \vee B) \leftrightarrow \neg A \wedge \neg B$	$(A \to B) \to \neg A \vee B$
$(\neg A \vee \neg B) \to \neg(A \wedge B)$	$((A \to B) \to A) \to A$
$(A \to B) \to (\neg B \to \neg A)$	

A pattern is visible: the constructive laws produce negations, and the classical ones produce disjunctions or extract a positive fact from negative information.

What you gain: disjunction and existence

Here is the payoff, which an introduction that only lists failures hides.

- **Disjunction property.** If $\vdash A \vee B$ is derivable intuitionistically (with no assumptions), then $\vdash A$ or $\vdash B$ is derivable.
- **Existence property.** If $\vdash \exists x.\, A(x)$ is derivable in intuitionistic predicate logic, then $\vdash A(t)$ is derivable for some term t.

Both follow from normalization (Chapter 4). A normal derivation with no assumptions must end with an introduction rule, by the same argument that proved consistency: an elimination at the root would lead up a chain of eliminations to an assumption, and there are none. For $A \vee B$ that introduction is $\vee$I1 or $\vee$I2, and its premise is a proof of the chosen side. You will meet this argument again in Chapter 9 as the *canonical forms* lemma: a closed value of a type has the shape that the type's introduction rules allow (a value of type `Bool` is `true` or `false`).

Classical logic has neither property. $\vdash A \vee \neg A$ is classically derivable for an atom A, but neither $\vdash A$ nor $\vdash \neg A$ is (each is false in some row of the truth table). A classical proof of a disjunction may carry no information about which side holds, and a classical existence proof may contain no witness. Compare two Lean proofs:

```
/-- A constructive existence proof contains its witness. -/
theorem exists_square : ∃ n : Nat, n * n = 49 := ⟨7, rfl⟩

/-- The drinker: in a nonempty pub there is someone such that, if they drink,
everyone drinks. The proof asks whether everyone drinks, a question it cannot
answer by computation, and so it never says who the person is. -/
theorem drinker {α : Type} [Inhabited α] (drinks : α → Prop) :
    ∃ x, drinks x → ∀ y, drinks y := by
  by_cases h : ∀ y, drinks y
  · exact ⟨default, fun _ => h⟩       -- anyone will do
  · obtain ⟨y, hy⟩ := Classical.not_forall.mp h   -- someone does not drink
    exact ⟨y, fun hdy => absurd hdy hy⟩

/-- info: 'Ch05.exists_square' does not depend on any axioms -/
#guard_msgs in
#print axioms exists_square
```

```
/-- info: 'Ch05.drinker' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms drinker
```

`exists_square` contains its witness, 7, and uses no axioms. `drinker` is a famous puzzle, usually called the drinker paradox: in any nonempty pub there is someone such that, if they drink, everyone drinks. The proof asks whether everyone drinks. If so, anyone will do; if not, `Classical.not_forall` (a classical lemma: from "not everyone drinks", someone who does not) produces a non-drinker, who satisfies the claim vacuously. The proof is correct, and it never tells you who the person is, because the question it asked has no computable answer.

This is precisely why intuitionistic logic is the logic of programming, and why every proof assistant with computational content (Lean's core, Rocq, Agda) is built on it. A program that has computed a value of type `Result<T, E>` knows which variant it holds; a program must *have* the `T` to return it. If you want to run a proof, the proof needs the witness.

> **NOTE**
>
> In Lean, `∃` and `∨` live in `Prop`, where proofs are not meant to be run, and Lean deliberately forbids extracting the witness of an `∃` into ordinary data (Chapter 10 explains this *elimination restriction*). The informative versions are types of data: `Σ` and subtypes for existence, `Sum` and `Decidable` for disjunction. We meet `Decidable` below.

5.4 Classical logic is not wrong

Two facts keep the story balanced.

The double-negation translation

The first is a theorem of Glivenko:

- **Glivenko's theorem.** A propositional formula A is a classical tautology if and only if $\neg\neg A$ is intuitionistically derivable.

So classical propositional logic *embeds* into intuitionistic logic: put $\neg\neg$ in front and every classical theorem becomes an intuitionistic one. Gödel and Gentzen extended the idea to predicate logic with a translation that inserts double negations more carefully (in front of atoms, disjunctions and existentials). Nothing is lost in terms of what can be proved. What changes is what a proof *gives you*: a proof of $\neg\neg A$ is weaker information than a proof of A.

The most important instance is $\neg\neg(A \vee \neg A)$. Here is its proof in tactic mode:

```
/-- Excluded middle cannot be refuted. The refutation `k` is called twice. -/
theorem nn_em (p : Prop) : ¬¬(p ∨ ¬p) := by
  intro k              -- k : ¬(p ∨ ¬p); goal: False
  apply k              -- goal: p ∨ ¬p
  right                -- commit to the right disjunct; goal: ¬p
  intro hp             -- hp : p; goal: False
  apply k              -- use k a second time; goal: p ∨ ¬p
```

```
  left                -- now we can commit to the left disjunct
  exact hp
```

Follow the goals. We must prove `¬¬(p ∨ ¬p)`, which unfolds to `¬(p ∨ ¬p) → False`, so `intro k` gives us a refutation `k` of excluded middle and leaves the goal `False`. The only way to produce `False` is to use `k`, so `apply k`, and now we must prove `p ∨ ¬p`. We cannot know whether `p` holds, so we commit to the right side, `¬p`, which is a promise: "give me a `p` and I will produce `False`". `intro hp` accepts such a `p`. Now we have a proof of `p` in hand, so we go back to `k` a second time and give it the left side, `Or.inl hp`. The term is short:

```
/--
info: theorem Ch05.nn_em : ∀ (p : Prop), ¬¬(p ∨ ¬p) :=
fun p k => k (Or.inr fun hp => k (Or.inl hp))
-/
#guard_msgs in
#print nn_em
```

The proof answers "¬p", and if that answer is ever challenged with an actual proof of `p`, it changes its mind and answers "p" using the very proof that was used to challenge it. Notice that `k` is used twice. That will matter when we write the same program in Rust.

Double negation is a monad

The second fact explains Glivenko's theorem computationally. Double negation behaves like a *monad*, a structure you will meet properly in Chapter 20: there is a way to put a value in (`pure`) and a way to chain computations (`bind`):

```
/-- `pure` for double negation: the same term as `dni`. -/
theorem nn_pure {p : Prop} (hp : p) : ¬¬p :=
  fun k => k hp

/-- `bind` for double negation: to refute a refutation of `q`, run the
double-negated `p` with a continuation that feeds each `p` to `f`. -/
theorem nn_bind {p q : Prop} (nnp : ¬¬p) (f : p → ¬¬q) : ¬¬q :=
  fun k => nnp (fun hp => f hp k)

/-- Inside `¬¬`, excluded middle is available, so the classical De Morgan law
holds there, with no axioms. -/
theorem nn_not_and_or (p q : Prop) : ¬¬(¬(p ∧ q) → ¬p ∨ ¬q) := by
  apply nn_bind (nn_em p)   -- borrow `p ∨ ¬p` inside the double negation
  intro hem
  apply nn_pure             -- now prove the formula itself
  intro h
  cases hem with
  | inl hp => exact Or.inr (fun hq => h ⟨hp, hq⟩)
  | inr hnp => exact Or.inl hnp

/-- info: 'Ch05.nn_not_and_or' does not depend on any axioms -/
#guard_msgs in
#print axioms nn_not_and_or
```

`nn_pure` is `dni` again. `nn_bind` says: to refute a refutation `k` of `q`, run the double-negated `p` with a continuation that sends each `p` to `f`, and passes `k` along. The payoff is `nn_not_and_or`: inside the double negation we may borrow `p ∨ ¬p` from `nn_em` and then reason classically, and the result is a constructive proof of the double negation of the classical De Morgan law, with no axioms. The same trick gives the double negations of the other classical laws:

```
/-- The double negations of the other classical laws are constructive too. -/
theorem nn_dne (p : Prop) : ¬¬(¬¬p → p) :=
  nn_bind (nn_em p) fun hem =>
    nn_pure fun hnnp => hem.elim id (fun hnp => absurd hnp hnnp)

theorem nn_peirce (p q : Prop) : ¬¬(((p → q) → p) → p) :=
  nn_bind (nn_em p) fun hem =>
    nn_pure fun h => hem.elim id (fun hnp => h (fun hp => absurd hp hnp))
```

Programmers know $\neg\neg A$ under another name. Replace $\bot$ by an arbitrary answer type R: then $(A \to R) \to R$ is a value in **continuation-passing style**, a computation that, instead of returning an A, takes a consumer of As. Double-negation elimination would be a way to get the A out of a continuation-passing computation without supplying a consumer, which is impossible in a pure language. Timothy Griffin observed that classical logic corresponds to programs with first-class continuations: Peirce's law is the type of the control operator `call/cc`, and a classical proof by contradiction is a program that captures its continuation and jumps back to it. Classical logic is perfectly computational; it computes with control effects rather than with pure functions.

We can run `nn_em` in Rust, with `Void` replaced by an answer type so that the program can actually return something. The full program is `code/rust/examples/ch05_negation.rs`:

```
/// A sum type: a value of `A ∨ B` says which side it is on.
enum Either<L, R> {
    Left(L),
    Right(R),
}

/// "Refutations" relative to an answer type `R`: consumers of `A`s. With
/// `R = Void` this is `¬A`; with an ordinary `R` we can run it.
type Neg<A, R> = Rc<dyn Fn(A) -> R>;

/// `¬¬(A ∨ ¬A)`, the double negation of excluded middle. Given a consumer `k`
/// of `A ∨ ¬A`, first answer "¬A" with a refutation. If `k` ever applies that
/// refutation to some `a`, go back and call `k` again, this time with
/// `Left(a)`. The consumer is used twice, which is why it is an `Rc<dyn Fn>`.
fn nn_em<A: 'static, R: 'static>(k: Neg<Either<A, Neg<A, R>>, R>) -> R {
    let k_again = Rc::clone(&k);
    let refutation: Neg<A, R> = Rc::new(move |a| k_again(Either::Left(a)));
    k(Either::Right(refutation))
}
```

`Neg<A, R>` is a consumer of `A`s producing an `R`. The consumer `k` is called twice, so it must be shareable: an `Rc<dyn Fn>` rather than a `Box<dyn FnOnce>`. In the logic of Chapter 4, using an assumption twice is free; in Rust it costs an `Rc`, a first glimpse of Chapter 6. Here is a consumer that, when handed a refutation, tests it on the number 42:

```
// Run ¬¬(A ∨ ¬A) with A = u32 and answers of type String.
let log = Rc::new(RefCell::new(Vec::new()));
let seen = Rc::clone(&log);
let k: Neg<Either<u32, Neg<u32, String>>, String> =
    Rc::new(move |answer| match answer {
        Either::Left(a) => {
            seen.borrow_mut().push(format!("k received Left({a})"));
            format!("A holds, witness {a}")
        }
        Either::Right(refutation) => {
            seen.borrow_mut()
                .push("k received Right(refutation)".to_string());
            seen.borrow_mut()
```

```
                .push("k applies the refutation to 42".to_string());
            refutation(42)
        }
    });
let result = nn_em(k);
for line in log.borrow().iter() {
    println!("{line}");
}
println!("result: {result}");
```

```
k received Right(refutation)
k applies the refutation to 42
k received Left(42)
result: A holds, witness 42
```

The trace is the "change of mind" in action: `k` first receives `Right(refutation)`, applies the refutation to 42, and the refutation calls `k` again with `Left(42)`. The final answer comes from the second call.

5.5 Negation, truth and falsity

Three states of knowledge

Classically, every proposition is true or false. Intuitionistically there are three situations, and conflating them is the commonest error in reading intuitionistic statements:

Situation	Meaning	Rust analogy
$\vdash A$	we have a construction of A	a value of type `A` exists
$\vdash \neg A$	we have a refutation of A	a function `A -> Void` exists
neither	A is open: no judgment can be asserted	we know nothing

$\neg A$ is a positive claim that needs positive evidence: a method that turns any proof of A into absurdity, which amounts to showing that A has no proof. "A is not provable" is a different statement altogether, a fact *about* the logic (like "excluded middle has no derivation"), which cannot even be written as a formula inside it. Excluded middle would say that the third row is always empty.

Empty types in Rust

The Rust side of negation is the empty type. An `enum` with no variants has no values, and matching on it needs no arms:

```
/// Falsity: a type with no values at all.
#[derive(Debug)]
enum Void {}

/// Rule ⊥E: from a value of the empty type, a value of any type. The `match`
/// has no arms because there is no case to handle.
fn absurd<T>(v: Void) -> T {
    match v {}
}
```

A refutation is a function into `Void`. Here are three of the constructive laws as Rust functions, plus $\neg\bot$:

```
/// `¬A`: a refutation of `A`, a function that turns any `A` into a `Void`.
type Not<A> = Box<dyn FnOnce(A) -> Void>;
```

```
/// `A → ¬¬A`: given an `a`, refute every refutation of `A` by feeding it `a`.
fn dni<A: 'static>(a: A) -> Not<Not<A>> {
    Box::new(move |refutation: Not<A>| refutation(a))
}

/// `(A → B) → (¬B → ¬A)`: contraposition is composition.
fn contrapose<A: 'static, B: 'static>(
    f: impl FnOnce(A) -> B + 'static,
    not_b: Not<B>,
) -> Not<A> {
    Box::new(move |a| not_b(f(a)))
}

/// `¬¬¬A → ¬A`: triple negation collapses.
fn tne<A: 'static>(nnn: Not<Not<Not<A>>>) -> Not<A> {
    Box::new(move |a| nnn(dni(a)))
}

/// `¬⊥`: falsity is refutable, by the identity function.
fn not_false() -> Not<Void> {
    Box::new(|v| v)
}
```

These functions can be built but never run to completion, since that would produce a `Void`; the example only constructs them, and the compiler's acceptance is the point. (`Box<dyn FnOnce>` is used because each refutation is called at most once; `nn_em` above needed more.)

The standard library has its own empty type, `std::convert::Infallible`, used for operations that cannot fail. The compiler knows that empty types have no values and uses it: a pattern for the impossible case can be omitted.

```
/// A name can be parsed from any string, so parsing cannot fail: the error
/// type is `Infallible`, the standard library's empty enum.
struct Name(String);

impl FromStr for Name {
    type Err = Infallible;
    fn from_str(s: &str) -> Result<Name, Infallible> {
        Ok(Name(s.trim().to_lowercase()))
    }
}

fn parse_name(s: &str) -> Name {
    // The pattern `Ok(name)` is irrefutable: `Err(e)` would need an `e` of an
    // empty type, so the compiler knows that case cannot occur.
    let Ok(name) = s.parse::<Name>();
    name
}

/// An `Option` of an empty type can only be `None`, and the match needs no
/// `Some` arm.
fn only_none(x: Option<Void>) -> &'static str {
    match x {
        None => "None",
    }
}
```

`let Ok(name) = s.parse::<Name>();` is accepted as an irrefutable pattern because the `Err` variant would need a value of type `Infallible`. This is ⊥E used by the exhaustiveness checker: the `Err` case is handled "by absurdity", with zero arms. Finally, Rust's built-in empty type is the never type `!`:

```rust
/// `!` is Rust's built-in empty type. A function returning `!` never returns,
/// and a `!` can stand in for a value of any type (⊥E, built into the language).
fn give_up(reason: &str) -> ! {
    panic!("giving up: {reason}")
}

fn digit_value(c: char) -> u32 {
    match c.to_digit(10) {
        Some(d) => d,
        None => give_up("not a digit"), // has type `!`, coerced to `u32`
    }
}
```

```
size_of::<Void>() = 0
size_of::<Infallible>() = 0
parse_name("  Ada ").0 = "ada"
only_none(None) = None
digit_value('7') = 7
```

Empty types have size zero: no bits are needed to store a value that never exists. The never type coerces to `u32` in `digit_value`, which is ⊥E built into the language. The difference from `Void` is that `!` is also the type of divergence: `loop {}` and `panic!()` have type `!`, so in Rust the empty type *does* have "proofs", and Rust is inconsistent as a logic. Chapter 10 returns to this.

5.6 Decidable propositions in Lean

Lean draws the line between constructive and classical reasoning with a class. `Decidable p` is a *type of data* recording which of `p` and `¬p` holds, together with the proof:

```
/--
info: inductive Decidable : Prop → Type
number of parameters: 1
constructors:
Decidable.isFalse : {p : Prop} → ¬p → Decidable p
Decidable.isTrue : {p : Prop} → p → Decidable p
-/
#guard_msgs in
#print Decidable
```

Compare with `p ∨ ¬p`. Both have two cases, but `Or` is a `Prop`, whose proofs Lean erases when it compiles, while `Decidable p` lives in `Type`: a value of it is a bit of information a program can branch on. `Decidable p` is the BHK reading of excluded middle for one particular `p`, and an instance of the class is a decision procedure. That is why `if` on a proposition needs one:

```
/-- `if` on a proposition needs a `Decidable` instance: a procedure that
says which side holds. For `n % 2 = 0` on `Nat`, Lean has one. -/
def parity (n : Nat) : String :=
  if n % 2 = 0 then "even" else "odd"

/-- info: "odd" -/
#guard_msgs in
#eval parity 7

-- With an arbitrary `p`, there is no procedure, and Lean says so.
/--
error: failed to synthesize instance of type class
  Decidable p

Hint: Type class instance resolution failures can be inspected with the `set_option
```

```
trace.Meta.synthInstance true` command.
-/
#guard_msgs in
def choose (p : Prop) : Nat := if p then 1 else 0

/-- Asking for the instance as an argument moves the obligation to the caller. -/
def choose' (p : Prop) [Decidable p] : Nat := if p then 1 else 0

/-- info: 1 -/
#guard_msgs in
#eval choose' (3 < 5)
```

Equality of natural numbers is decidable, so `parity` compiles and runs. For an arbitrary `p`, Lean cannot synthesize an instance: there is no procedure that decides every proposition. `choose'` asks its caller for the instance instead, and `choose' (3 < 5)` gets one from the library. The `decide` tactic that you have used since Chapter 1 works the same way: it evaluates the `Decidable` instance of the goal and succeeds if the answer is `isTrue`.

Classical logic offers a way out, at a price:

```
/-- `Classical.propDecidable` makes every proposition "decidable" by the
axiom of choice, but the result cannot be run: there is no code behind it. -/
noncomputable def chooseClassical (p : Prop) : Nat :=
  have := Classical.propDecidable p
  if p then 1 else 0
```

`Classical.propDecidable` produces a `Decidable p` for every `p`, using `Classical.choice`. The definition type-checks, but Lean requires the keyword `noncomputable`: there is no code to generate, because the axiom of choice has no algorithm behind it. In Lean the boundary between constructive and classical mathematics is not a matter of style; it is the boundary between what can be run and what cannot.

5.7 Kripke semantics

Truth tables are too coarse for intuitionistic logic: excluded middle is true in every row. Saul Kripke's semantics, from the 1960s, replaces the single row with a *process* of acquiring knowledge.

Worlds and forcing

A **Kripke model** consists of:

- a set W of **worlds**, thought of as states of knowledge;
- a preorder $\leq$ on W (reflexive and transitive), where $w \leq w'$ means "w' is a possible future of w, in which we know at least as much";
- for each world, the set of atoms known at that world, which must be **monotone**: once known, an atom stays known in every future.

The **forcing** relation $w \Vdash A$, "at w we know A", is defined by recursion on A:

- $w \Vdash P$ iff the atom P is known at w;
- $w \Vdash \top$ always, and $w \Vdash \bot$ never;
- $w \Vdash A \wedge B$ iff $w \Vdash A$ and $w \Vdash B$;
- $w \Vdash A \vee B$ iff $w \Vdash A$ or $w \Vdash B$;

- $w \Vdash A \to B$ iff for every $w' \geq w$, if $w' \Vdash A$ then $w' \Vdash B$.

The clauses for $\wedge$, $\vee$, $\top$ and $\bot$ are those of truth tables, evaluated at one world. Implication is the only clause that looks at other worlds: asserting $A \to B$ now is a commitment that holds *whatever we learn later*. For negation this gives

$$w \Vdash \neg A \quad \text{iff} \quad \text{no} \quad w' \geq w \quad \text{forces} \quad A$$

$\neg A$ holds when A is refuted now *and will stay refuted forever*. That is much stronger than "A is not known yet", and the gap between the two is where intuitionistic logic lives. Similarly, $w \Vdash \neg\neg A$ iff every future of w can be extended to a world that forces A: A is not available yet, but it can never be ruled out.

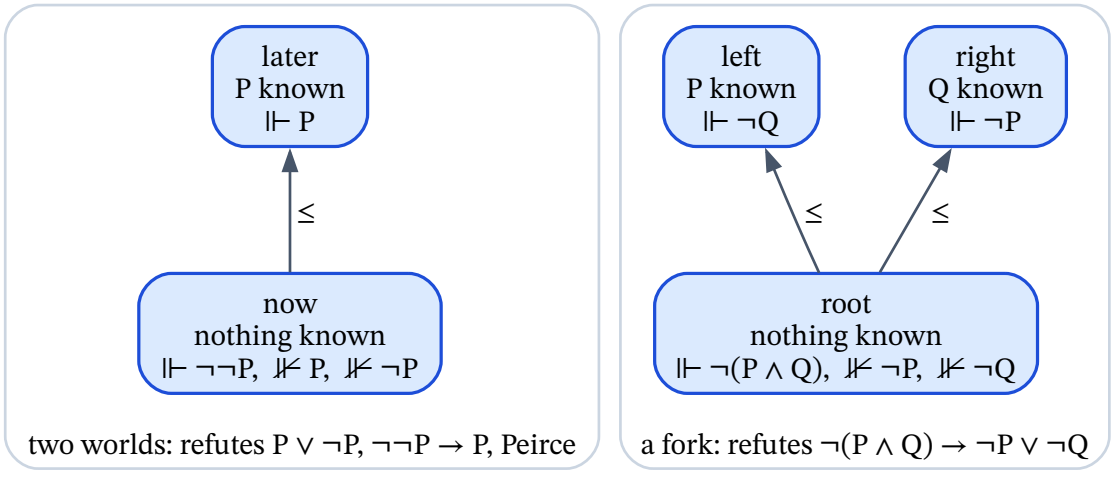
A sequent $\Gamma \vdash A$ is **valid** if in every Kripke model, every world that forces all formulas of Γ forces A. Two theorems connect the semantics with the rules of Chapter 4:

- **Soundness.** Every derivable sequent is valid.
- **Completeness** (Kripke). Every valid sequent is derivable.

We will prove soundness in Lean. Completeness is harder and we only state it; the Rust search below gives evidence for it on small examples.

A two-world countermodel

Take two worlds, *now* and *later*, with $now \leq later$. Nothing is known now; later, the atom P has been established.



Two Kripke countermodels. Left: a two-world model in which P becomes known later; it refutes excluded middle, double-negation elimination and Peirce's law at "now". Right: a fork in which P and Q become known on different branches; it refutes the De Morgan law.

Check excluded middle at *now*:

- $now \nVdash P$, because P is not known yet;
- $now \nVdash \neg P$, because a future world, *later*, forces P;

- so $now \nVdash P \vee \neg P$.

That is the whole counterexample, and the whole intuition: P is undecided now and becomes true later, so neither disjunct can be asserted now. The same model refutes double-negation elimination: every future of *now* can reach a world forcing P, so $now \Vdash \neg\neg P$, yet $now \nVdash P$. By soundness, none of these formulas is derivable.

Notice what happens with a single world: the quantifier "for every $w' \geq w$" ranges over w alone, and the clause for $\to$ becomes the truth table. A one-world Kripke model *is* a row of a truth table, and classical logic is intuitionistic logic in a world where nothing more can ever be learned.

Kripke models in Lean

The file `code/lean/Book/Ch05/Kripke.lean` imports the deep embedding of Chapter 4 (`Book.Ch04.Deep`, from the same pair of chapters) and gives it a Kripke semantics. A model is a structure over a type of worlds with a preorder, which we take from Mathlib's class `Preorder` (it provides `≤`, `le_refl` and `le_trans`):

```
/-- A Kripke model over a preordered set of worlds `W`. Read `w ≤ w'` as
"`w'` is a possible future of `w`, where we know at least as much", and
`val w n` as "at world `w`, atom `n` is known to hold". Knowledge is never
lost: that is `mono`. -/
structure KripkeModel (W : Type) [Preorder W] where
  val : W → Nat → Bool
  mono : ∀ {w w' : W} {n : Nat}, w ≤ w' → val w n = true → val w' n = true
```

```
/-- The forcing relation `w ⊩ A`. Only implication looks at other worlds: it
must hold in every future, not just now. -/
def Forces (M : KripkeModel W) : W → Formula → Prop
  | w, .var n   => M.val w n = true
  | _, .top     => True
  | _, .bot     => False
  | w, .and A B => Forces M w A ∧ Forces M w B
  | w, .or A B  => Forces M w A ∨ Forces M w B
  | w, .imp A B => ∀ w', w ≤ w' → Forces M w' A → Forces M w' B
```

`Forces` is a `Prop`-valued function defined by recursion on the formula, one equation per clause. The negation clause comes out as expected:

```
/-- Unfolding the definition for negation: `w ⊩ ¬A` means that no future of
`w` forces `A`. -/
theorem forces_neg (M : KripkeModel W) (w : W) (A : Formula) :
    Forces M w A.neg ↔ ∀ w', w ≤ w' → ¬ Forces M w' A := by
  simp [Formula.neg, Forces]
```

The key lemma is monotonicity: forcing is never lost.

```
/-- **Monotonicity.** What is forced stays forced in every future world. By
induction on the formula; the case of implication uses transitivity. -/
theorem Forces.mono (M : KripkeModel W) {w w' : W} (hle : w ≤ w') :
    ∀ A : Formula, Forces M w A → Forces M w' A
  | .var _, h => M.mono hle h
  | .top, _ => trivial
  | .bot, h => h
  | .and A B, ⟨hA, hB⟩ => ⟨Forces.mono M hle A hA, Forces.mono M hle B hB⟩
  | .or A _, .inl hA => .inl (Forces.mono M hle A hA)
  | .or _ B, .inr hB => .inr (Forces.mono M hle B hB)
  | .imp _ _, h => fun w'' hle' hA => h w'' (le_trans hle hle') hA
```

This proof is a recursive function, defined by pattern matching on the formula and the proof of forcing, and Lean checks that every recursive call is on a smaller formula. Atoms are monotone by the model's `mono` field. For a conjunction, both parts move forward; for a disjunction, whichever part holds moves forward. For an implication, `h` promises B in every future of w that forces A; a future w'' of w' is also a future of w, by `le_trans`. That is where transitivity is needed.

Soundness is proved by induction on derivations, like the truth-table version of Chapter 4:

```
/-- **Soundness.** A derivable sequent is forced at every world of every
Kripke model that forces all of its assumptions. -/
theorem kripke_soundness {Γ : List Formula} {A : Formula} (h : Proves Γ A) :
    ∀ {W : Type} [Preorder W] (M : KripkeModel W) (w : W),
      (∀ B ∈ Γ, Forces M w B) → Forces M w A := by
  induction h with
  | hyp hmem => intro W _ M w hΓ; exact hΓ _ hmem
  | topI => intro W _ M w _; trivial
  | botE _ ih => intro W _ M w hΓ; exact (ih M w hΓ).elim
  | andI _ _ ih₁ ih₂ => intro W _ M w hΓ; exact ⟨ih₁ M w hΓ, ih₂ M w hΓ⟩
  | andE1 _ ih => intro W _ M w hΓ; exact (ih M w hΓ).1
  | andE2 _ ih => intro W _ M w hΓ; exact (ih M w hΓ).2
  | orI1 _ ih => intro W _ M w hΓ; exact .inl (ih M w hΓ)
  | orI2 _ ih => intro W _ M w hΓ; exact .inr (ih M w hΓ)
  | orE _ _ _ ih ihA ihB =>
    intro W _ M w hΓ
    rcases ih M w hΓ with hA | hB
    · apply ihA M w
      intro C hC
      rcases List.mem_cons.mp hC with rfl | hC
      · exact hA
      · exact hΓ C hC
    · apply ihB M w
      intro C hC
      rcases List.mem_cons.mp hC with rfl | hC
      · exact hB
      · exact hΓ C hC
  | impI _ ih =>
    -- To force A → B at w, take any future w' forcing A. The assumptions,
    -- forced at w, are still forced at w' (monotonicity), so the premise
    -- applies at w' with A added to the context.
    intro W _ M w hΓ w' hle hA
    apply ih M w'
    intro C hC
    rcases List.mem_cons.mp hC with rfl | hC
    · exact hA
    · exact Forces.mono M hle C (hΓ C hC)
  | impE _ _ ih₁ ih₂ =>
    -- Use the implication at w itself, which is its own future (w ≤ w).
    intro W _ M w hΓ
    exact ih₁ M w hΓ w (le_refl w) (ih₂ M w hΓ)
```

Most cases are as before. The two interesting ones are the rules for implication. For **impI** we must show that w forces $A \to B$: take any future w' with `hle : w ≤ w'` that forces A. The induction hypothesis applies at w' with context `A :: Γ`, so we must show that w' forces everything in it: A by assumption, and each formula of `Γ` because it was forced at w and forcing is monotone. Without monotonicity, the proof would break exactly here. For **impE** we use the implication at w itself, which is its own future by `le_refl`.

Deciding forcing with decide

For a model with finitely many worlds, forcing is decidable: each quantifier "for every w'" ranges over a finite list. We say so with an instance. First a small class for types whose elements can be listed:

```
/-- A type whose elements can be listed. (Mathlib's `Fintype` says the same,
but checking its instances by `decide` drags classical lemmas into the proof;
this small class keeps every result in this file constructive.) -/
class Enumerable (W : Type) where
  elems : List W
  complete : ∀ w, w ∈ elems

/-- Over an enumerable type, "for all `w`" is decidable: check every element
of the list. -/
instance decForallEnum {W : Type} [Enumerable W] (p : W → Prop) [DecidablePred p] :
    Decidable (∀ w, p w) :=
  decidable_of_iff (∀ w ∈ Enumerable.elems, p w)
    ⟨fun h w => h w (Enumerable.complete w), fun h w _ => h w⟩
```

`decidable_of_iff` builds a decision procedure for `∀ w, p w` from one for the equivalent `∀ w ∈ elems, p w`, which core Lean can decide by checking the list. Then forcing is decidable by recursion on the formula:

```
/-- In a finite model with a decidable order, forcing is decidable: every
quantifier ranges over finitely many worlds. -/
instance decForces [Enumerable W] [DecidableRel (α := W) (· ≤ ·)] (M : KripkeModel W) :
    ∀ (w : W) (A : Formula), Decidable (Forces M w A)
  | w, .var n => inferInstanceAs (Decidable (M.val w n = true))
  | _, .top => inferInstanceAs (Decidable True)
  | _, .bot => inferInstanceAs (Decidable False)
  | w, .and A B =>
    have := decForces M w A
    have := decForces M w B
    inferInstanceAs (Decidable (Forces M w A ∧ Forces M w B))
  | w, .or A B =>
    have := decForces M w A
    have := decForces M w B
    inferInstanceAs (Decidable (Forces M w A ∨ Forces M w B))
  | w, .imp A B =>
    have := fun w' => decForces M w' A
    have := fun w' => decForces M w' B
    inferInstanceAs (Decidable (∀ w', w ≤ w' → Forces M w' A → Forces M w' B))
```

Each case asks Lean to find the instance for the unfolded clause (`inferInstanceAs`), after making the instances for the subformulas available with `have`. Now the two-world model. The worlds are a two-element type with its order:

```
/-- Two stages of knowledge: `now`, and `later`, when we may know more. -/
inductive Stage where
  | now | later
  deriving DecidableEq

/-- `now ≤ later`, and each stage is below itself. -/
def Stage.le : Stage → Stage → Bool
  | .now, _ => true
  | .later, .later => true
  | .later, .now => false

instance : Preorder Stage where
  le a b := Stage.le a b = true
  le_refl a := by cases a <;> rfl
  le_trans a b c := by cases a <;> cases b <;> cases c <;> decide

instance : DecidableRel (α := Stage) (· ≤ ·) :=
  fun a b => inferInstanceAs (Decidable (Stage.le a b = true))

instance : Enumerable Stage where
  elems := [.now, .later]
  complete w := by cases w <;> decide
```

```
/-- Nothing is known `now`; `later`, the atom `P` (number 0) has been
established. -/
def twoWorlds : KripkeModel Stage where
  val
    | .later, 0 => true
    | _, _ => false
  mono {w w' n} hle hv := by
    change Stage.le w w' = true at hle
    cases w <;> cases w' <;> simp_all [Stage.le]
```

The valuation is a pattern match: atom 0 (`P`) is known `later` , nothing else anywhere. Monotonicity is proved by checking every pair of worlds. And now `decide` checks the countermodel, clause by clause, by evaluating the decision procedure:

```
/-- `P` is not forced now, but it will be. -/
example : ¬ Forces twoWorlds .now P ∧ Forces twoWorlds .later P := by decide

/-- `¬P` is not forced now either: a future world forces `P`. -/
example : ¬ Forces twoWorlds .now P.neg := by decide

/-- So excluded middle fails now. -/
theorem lem_fails : ¬ Forces twoWorlds .now (.or P P.neg) := by decide

/-- `¬¬P` holds now (`P` is never refuted), but `¬¬P → P` does not:
double-negation elimination fails. -/
theorem dne_fails : Forces twoWorlds .now P.neg.neg ∧
    ¬ Forces twoWorlds .now (.imp P.neg.neg P) := by decide

/-- Peirce's law `((P → Q) → P) → P` fails too (`Q` is never known). -/
theorem peirce_fails : ¬ Forces twoWorlds .now (.imp (.imp (.imp P Q) P) P) := by decide
```

With soundness, each failure becomes a statement about the proof system of Chapter 4:

```
/-- Excluded middle is not derivable in the natural deduction of Chapter 4:
if it were, soundness would make it forced at `now`. -/
theorem lem_not_derivable : ¬ Proves [] (.or P P.neg) := by
  intro h
  exact lem_fails (kripke_soundness h twoWorlds .now (by simp))

theorem dne_not_derivable : ¬ Proves [] (.imp P.neg.neg P) := by
  intro h
  exact dne_fails.2 (kripke_soundness h twoWorlds .now (by simp))

theorem peirce_not_derivable : ¬ Proves [] (.imp (.imp (.imp P Q) P) P) := by
  intro h
  exact peirce_fails (kripke_soundness h twoWorlds .now (by simp))
```

If excluded middle had a derivation `h` , then `kripke_soundness h` would make it forced at `now` in `twoWorlds` (the empty context is trivially forced, which `simp` checks), contradicting `lem_fails` . This is the theorem that Chapter 4 could not prove with truth tables. And yet:

```
/-- Yet all three are tautologies: truth tables cannot tell them apart from
the derivable formulas. -/
example : isTautology 1 (.or P P.neg) = true ∧
    isTautology 1 (.imp P.neg.neg P) = true ∧
    isTautology 2 (.imp (.imp (.imp P Q) P) P) = true := by decide
```

all three are tautologies. And the proof that excluded middle is not derivable is itself constructive (`propext` and `Quot.sound` come from `simp` ; there is no `Classical.choice`):

```
-- The proof that excluded middle is not derivable is itself constructive.
/-- info: 'Ch05.lem_not_derivable' depends on axioms: [propext, Quot.sound] -/
```

```
#guard_msgs in
#print axioms lem_not_derivable
```

Finally, the claim that classical logic is the one-world case is a theorem:

```
/-- The only possible order on a one-element type. -/
local instance : Preorder Unit where
  le _ _ := True
  le_refl _ := trivial
  le_trans _ _ _ _ _ := trivial

/-- A model with a single world. -/
def oneWorld (v : Nat → Bool) : KripkeModel Unit where
  val _ n := v n
  mono _ h := h

/-- In a one-world model, forcing is truth-table evaluation: the quantifier
over futures has only the present to range over. -/
theorem forces_oneWorld (v : Nat → Bool) :
    ∀ A : Formula, Forces (oneWorld v) () A ↔ eval v A = true
  | .var _ => Iff.rfl
  | .top => by simp [Forces, eval]
  | .bot => by simp [Forces, eval]
  | .and A B => by
    simp [Forces, eval, ← forces_oneWorld v A, ← forces_oneWorld v B]
  | .or A B => by
    simp [Forces, eval, ← forces_oneWorld v A, ← forces_oneWorld v B]
  | .imp A B => by
    have hA := forces_oneWorld v A
    have hB := forces_oneWorld v B
    simp only [Forces, eval]
    cases h : eval v A <;> simp_all
```

(`local instance` gives `Unit` its trivial order in this file only.) The proof goes formula by formula; for implication, the "for every future" clause has only the present to check, and `simp_all` reduces both sides to the same Boolean condition.

A fork

Two worlds in a line are not enough to refute every classical law. The De Morgan law $\neg(P \wedge Q) \to \neg P \vee \neg Q$ needs a world with two incomparable futures, as on the right of the figure above:

```
/-- Three worlds: a root and two incomparable futures. -/
inductive Fork where
  | root | left | right
  deriving DecidableEq

/-- The order: the root is below everything, and each world is below itself. -/
def Fork.le : Fork → Fork → Bool
  | .root, _ => true
  | .left, .left => true
  | .right, .right => true
  | _, _ => false

instance : Preorder Fork where
  le a b := Fork.le a b = true
  le_refl a := by cases a <;> rfl
  le_trans a b c := by cases a <;> cases b <;> cases c <;> decide

instance : DecidableRel (α := Fork) (· ≤ ·) :=
  fun a b => inferInstanceAs (Decidable (Fork.le a b = true))

instance : Enumerable Fork where
  elems := [.root, .left, .right]
  complete w := by cases w <;> decide
```

```
/-- `P` becomes known on the left branch, `Q` on the right branch. -/
def fork : KripkeModel Fork where
  val
    | .left, 0 => true
    | .right, 1 => true
    | _, _ => false
  mono {w w' n} hle hv := by
    change Fork.le w w' = true at hle
    cases w <;> cases w' <;> simp_all [Fork.le]
```

```
/-- The De Morgan law `¬(P ∧ Q) → ¬P ∨ ¬Q` fails at the root. -/
theorem de_morgan_fails :
    ¬ Forces fork .root (.imp (Formula.and P Q).neg (.or P.neg Q.neg)) := by decide

/-- Weak excluded middle `¬P ∨ ¬¬P` fails at the root. -/
theorem wlem_fails : ¬ Forces fork .root (.or P.neg P.neg.neg) := by decide

theorem de_morgan_not_derivable : ¬ Proves [] (.imp (Formula.and P Q).neg (.or P.neg
Q.neg)) :=
  fun h => de_morgan_fails (kripke_soundness h fork .root (by simp))
```

At the root, $P \wedge Q$ is refuted, because no world knows both. But $\neg P$ fails at the root (the left branch learns P) and $\neg Q$ fails too (the right branch learns Q). The root commits to the refutation of the conjunction without being able to say which conjunct is to blame.

Searching for countermodels in Rust

How did we know which models to try? We searched. The program `code/rust/examples/ch05_kripke.rs` enumerates every Kripke model with at most three worlds and two atoms, and for each formula looks for a world where it fails. Forcing is the definition, transcribed:

```
/// A finite Kripke model. Worlds are `0..n`; `le[w][v]` says that `v` is a
/// possible future of `w`; `val[atom][w]` says that the atom is known at `w`.
struct Model {
    n: usize,
    le: Vec<Vec<bool>>,
    val: Vec<Vec<bool>>,
}

/// The forcing relation `w ⊩ A`. Only implication (and so negation) looks
/// beyond the present world: it must hold in every future.
fn forces(m: &Model, w: usize, a: &Formula) -> bool {
    match a {
        Atom(i) => m.val[*i][w],
        Bot => false,
        Formula::And(a, b) => forces(m, w, a) && forces(m, w, b),
        Formula::Or(a, b) => forces(m, w, a) || forces(m, w, b),
        Formula::Imp(a, b) => (0..m.n)
            .filter(|&v| m.le[w][v])
            .all(|v| !forces(m, v, a) || forces(m, v, b)),
    }
}
```

The enumeration builds every partial order on n worlds (every subset of ordered pairs that is reflexive, antisymmetric and transitive) and every monotone valuation (every set of worlds closed upward, for each atom). Restricting to partial orders loses nothing: worlds that are below each other force the same formulas and can be merged.

```
/// Every partial order on `n` worlds: reflexive, antisymmetric, transitive.
fn partial_orders(n: usize) -> Vec<Vec<Vec<bool>>> {
```

```
    let pairs: Vec<(usize, usize)> = (0..n)
        .flat_map(|i| (0..n).map(move |j| (i, j)))
        .filter(|(i, j)| i != j)
        .collect();
    let mut orders = Vec::new();
    for bits in 0..1u32 << pairs.len() {
        let mut le = vec![vec![false; n]; n];
        for (i, row) in le.iter_mut().enumerate() {
            row[i] = true;
        }
        for (k, &(i, j)) in pairs.iter().enumerate() {
            le[i][j] = bits >> k & 1 == 1;
        }
        let antisymmetric = pairs.iter().all(|&(i, j)| !(le[i][j] && le[j][i]));
        let transitive = (0..n).all(|i| {
            (0..n).all(|j| (0..n).all(|k| !(le[i][j] && le[j][k]) || le[i][k]))
        });
        if antisymmetric && transitive {
            orders.push(le);
        }
    }
    orders
}

/// Every monotone valuation of one atom: a set of worlds closed upward
/// (once known, an atom stays known).
fn up_sets(le: &[Vec<bool>]) -> Vec<Vec<bool>> {
    let n = le.len();
    (0..1u32 << n)
        .map(|bits| (0..n).map(|w| bits >> w & 1 == 1).collect::<Vec<bool>>())
        .filter(|set| (0..n).all(|w| (0..n).all(|v| !(set[w] && le[w][v]) || set[v])))
        .collect()
}

/// Every model with 1 to `max_worlds` worlds and two atoms.
fn all_models(max_worlds: usize) -> Vec<Model> {
    let mut models = Vec::new();
    for n in 1..=max_worlds {
        for le in partial_orders(n) {
            let ups = up_sets(&le);
            for vp in &ups {
                for vq in &ups {
                    models.push(Model {
                        n,
                        le: le.clone(),
                        val: vec![vp.clone(), vq.clone()],
                    });
                }
            }
        }
    }
    models
}
```

```
for (name, law) in &classical {
    // `models` is ordered by size, so the first hit is a smallest one.
    let found = models
        .iter()
        .find_map(|m| (0..m.n).find(|&w| !forces(m, w, law)).map(|w| (m, w)));
    match found {
        Some((m, w)) => println!("{name:<17} {law}\n  {}", describe(m, w)),
        None => println!("{name:<17} {law}\n  no countermodel found"),
    }
}
```

```
models with at most 3 worlds and 2 atoms: 564

excluded middle   P v ¬P
```

```
  2 worlds, 0≤1, P@{1} Q@{}, fails at 0
double negation   ¬¬P → P
  2 worlds, 0≤1, P@{1} Q@{}, fails at 0
Peirce            ((P → Q) → P) → P
  2 worlds, 0≤1, P@{1} Q@{}, fails at 0
De Morgan         ¬(P ∧ Q) → ¬P ∨ ¬Q
  3 worlds, 0≤1 0≤2, P@{1} Q@{2}, fails at 0
weak excl. middle ¬P ∨ ¬¬P
  3 worlds, 0≤1 0≤2, P@{1} Q@{}, fails at 0
linearity         (P → Q) ∨ (Q → P)
  3 worlds, 0≤1 0≤2, P@{1} Q@{2}, fails at 0

holds in all 564 models: ¬¬(P ∨ ¬P)
holds in all 564 models: P → ¬¬P
holds in all 564 models: ¬¬¬P → ¬P
holds in all 564 models: ¬(P ∨ Q) → ¬P ∧ ¬Q
holds in all 564 models: ¬P ∨ ¬Q → ¬(P ∧ Q)
holds in all 564 models: (P → Q) ∧ (P → ¬Q) → ¬P
```

Read a line such as `2 worlds, 0≤1, P@{1} Q@{}, fails at 0` as "worlds 0 and 1 with $0 \leq 1$, P known at world 1, Q nowhere; the formula fails at world 0": our two-world model. Because the models are enumerated by size, each one reported is a smallest countermodel. Excluded middle, double-negation elimination and Peirce's law all fall to two worlds. The De Morgan law, weak excluded middle and the linearity axiom $(P \to Q) \vee (Q \to P)$ survive every model with two worlds and need the fork. That is the semantic shadow of the diagram of intermediate logics: the two-world chain validates the De Morgan law but refutes excluded middle, so the De Morgan law cannot imply excluded middle. (The search only tries atoms for P and Q, but in fact every instance of the De Morgan law holds on any frame in which two futures of a world always have a common future, as in a chain.) The six intuitionistic laws at the bottom hold in all 564 models, as soundness predicts. A search is evidence, not proof; the proofs are the Lean files.

5.8 Heyting algebras

Kripke models describe intuitionistic logic with worlds; the algebraic semantics describes it with an ordered set of truth values. For classical logic the truth values form a **Boolean algebra**: `true` and `false`, or more generally the subsets of a set, with $\cap$, $\cup$ and complement. Intuitionistic logic needs a weaker structure.

Definition

A **Heyting algebra** is a partially ordered set $(H, \leq)$ with

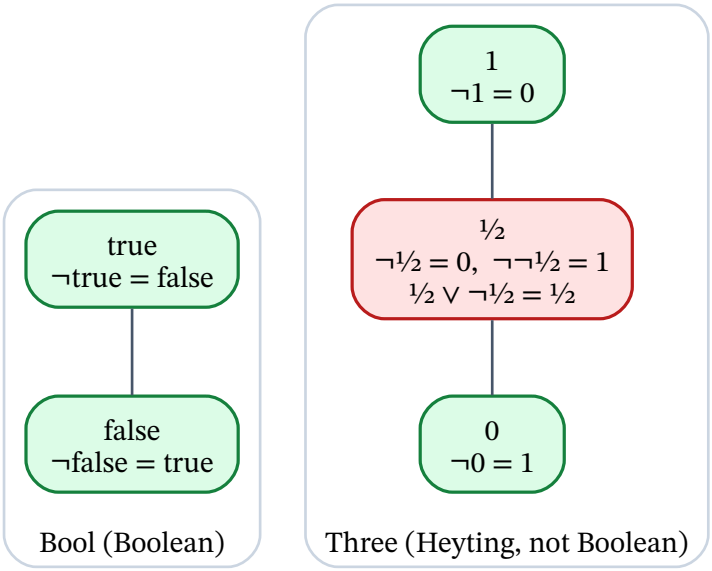
- a least element $\bot$ and a greatest element $\top$;
- binary meets $a \sqcap b$ (greatest lower bound, "and") and joins $a \sqcup b$ (least upper bound, "or");
- an implication $a \Rightarrow b$ characterized by **residuation**:

$$c \leq (a \Rightarrow b) \quad \text{iff} \quad c \sqcap a \leq b$$

Read $\leq$ as "entails". Residuation says that $a \Rightarrow b$ is the weakest element that, together with a, entails b: it is $\to$I and $\to$E rolled into one equivalence, exactly as $\Gamma, A \vdash B$ holds iff $\Gamma \vdash A \to B$. Negation is defined as in Chapter 4: $\neg a = a \Rightarrow \bot$, the largest element whose meet with a is $\bot$. In a Boolean

algebra, $a \Rightarrow b$ is $\neg a \sqcup b$ and $a \sqcup \neg a = \top$. In a Heyting algebra the second equation may fail: $\neg a$ is only a *pseudo-complement*.

The smallest example that is not Boolean is the three-element chain $0 < ½ < 1$: false, undecided, true. In a chain, meet is minimum, join is maximum, and $a \Rightarrow b$ is 1 if $a \leq b$ and b otherwise. Then $\neg\, ½ = ½ \Rightarrow 0 = 0$, so $½ \sqcup \neg\, ½ = ½ \neq 1$: excluded middle fails at the undecided value. And $\neg\neg\, ½ = \neg 0 = 1 \neq ½$: double-negation elimination fails too.



The two-element Boolean algebra and the three-element Heyting chain, with their negations. The middle element is its own witness against excluded middle.

A Heyting algebra class from scratch

In `code/lean/Book/Ch05/Heyting.lean` we write the definition as a Lean class. We borrow only the *notation* from Lean and Mathlib: the classes `LE`, `Min`, `Max`, `HImp`, `Top` and `Bot` carry the symbols `≤`, `⊓`, `⊔`, `⇨`, `⊤` and `⊥` and no laws at all. Every law is a field of our class:

```
/-- A Heyting algebra: a bounded lattice with an implication `⇨` that is
related to `⊓` by residuation. (Inside `namespace Ch05` this name shadows
Mathlib's `HeytingAlgebra`, which we meet at the end of the file.) -/
class HeytingAlgebra (H : Type) extends LE H, Min H, Max H, HImp H, Top H, Bot H where
  -- `≤` is a partial order
  le_refl : ∀ a : H, a ≤ a
  le_trans : ∀ a b c : H, a ≤ b → b ≤ c → a ≤ c
  le_antisymm : ∀ a b : H, a ≤ b → b ≤ a → a = b
  -- `a ⊓ b` is the greatest lower bound ("and")
  inf_le_left : ∀ a b : H, a ⊓ b ≤ a
  inf_le_right : ∀ a b : H, a ⊓ b ≤ b
  le_inf : ∀ a b c : H, a ≤ b → a ≤ c → a ≤ b ⊓ c
  -- `a ⊔ b` is the least upper bound ("or")
  le_sup_left : ∀ a b : H, a ≤ a ⊔ b
  le_sup_right : ∀ a b : H, b ≤ a ⊔ b
  sup_le : ∀ a b c : H, a ≤ c → b ≤ c → a ⊔ b ≤ c
  -- `⊥` is the least element ("false"), `⊤` the greatest ("true")
  bot_le : ∀ a : H, ⊥ ≤ a
  le_top : ∀ a : H, a ≤ ⊤
```

```
  -- residuation: `b ⇨ c` is the largest `a` with `a ⊓ b ≤ c` ("implies")
  le_himp_iff : ∀ a b c : H, a ≤ b ⇨ c ↔ a ⊓ b ≤ c
```

`extends` makes our class include the notation classes, like a Rust trait with supertraits. Mathlib has a class with the same name; inside `namespace Ch05` ours shadows it, and we refer to Mathlib's as `_root_.HeytingAlgebra` at the end of the file. Negation is a definition:

```
/-- Negation is implication into the bottom element. -/
def neg {H : Type} [HeytingAlgebra H] (a : H) : H := a ⇨ ⊥
```

From the axioms alone we can prove the laws of intuitionistic negation:

```
/-- Meet is commutative (as an inequality; equality follows by antisymmetry). -/
theorem inf_comm_le (a b : H) : a ⊓ b ≤ b ⊓ a :=
  le_inf _ _ _ (inf_le_right a b) (inf_le_left a b)

/-- Modus ponens: `(a ⇨ b) ⊓ a ≤ b`. Residuation, read from right to left,
applied to `a ⇨ b ≤ a ⇨ b`. -/
theorem himp_inf_le (a b : H) : (a ⇨ b) ⊓ a ≤ b :=
  (le_himp_iff _ _ _).mp (le_refl _)

/-- `a ≤ ¬¬a`: unfolding, `a ≤ (a ⇨ ⊥) ⇨ ⊥`, which by residuation is
`a ⊓ (a ⇨ ⊥) ≤ ⊥`, modus ponens after swapping the meet. -/
theorem le_neg_neg (a : H) : a ≤ neg (neg a) :=
  (le_himp_iff _ _ _).mpr
    (le_trans _ _ _ (inf_comm_le _ _) (himp_inf_le a ⊥))

/-- Negation reverses the order. -/
theorem neg_anti {a b : H} (h : a ≤ b) : neg b ≤ neg a :=
  (le_himp_iff _ _ _).mpr
    (le_trans _ _ _
      (le_inf _ _ _ (inf_le_left _ _) (le_trans _ _ _ (inf_le_right _ _) h))
      (himp_inf_le b ⊥))

/-- Triple negation collapses: `¬¬¬a = ¬a`. -/
theorem neg_neg_neg (a : H) : neg (neg (neg a)) = neg a :=
  le_antisymm _ _ (neg_anti (le_neg_neg a)) (le_neg_neg (neg a))

/-- Non-contradiction: `a ⊓ ¬a ≤ ⊥`. -/
theorem inf_neg_le_bot (a : H) : a ⊓ neg a ≤ ⊥ :=
  le_trans _ _ _ (inf_comm_le _ _) (himp_inf_le a ⊥)
```

These are term-mode proofs that chain the axioms, and they follow the logic closely. `himp_inf_le` is modus ponens: residuation read from right to left, applied to `a ⇨ b ≤ a ⇨ b`. `le_neg_neg` is $a \le \neg\neg a$, the algebraic `dni`: by residuation it suffices that $a \sqcap \neg a \le \bot$, which is modus ponens after swapping the meet. `neg_anti` says that negation reverses the order, and `neg_neg_neg` combines the two into the collapse of triple negation. (Inside `namespace HeytingAlgebra`, names such as `le_trans` refer to the fields of our class.) Residuation also implies that every Heyting algebra is distributive, which soundness for ∨E needs:

```
/-- Every Heyting algebra is distributive. By residuation it suffices to
show `a ⊔ b ≤ c ⇨ d` for `d = (a ⊓ c) ⊔ (b ⊓ c)`, and a join is below
something when both parts are. -/
theorem sup_inf_le (a b c : H) : (a ⊔ b) ⊓ c ≤ (a ⊓ c) ⊔ (b ⊓ c) := by
  apply (le_himp_iff _ _ _).mp
  apply sup_le
  · exact (le_himp_iff _ _ _).mpr (le_sup_left _ _)
  · exact (le_himp_iff _ _ _).mpr (le_sup_right _ _)
```

Soundness in every Heyting algebra

A formula is interpreted by recursion, and a context by the meet of its formulas:

```
/-- The value of a formula in a Heyting algebra, given a value for each atom. -/
def interp {H : Type} [HeytingAlgebra H] (f : Nat → H) : Formula → H
  | .var n   => f n
  | .top     => ⊤
  | .bot     => ⊥
  | .and A B => interp f A ⊓ interp f B
  | .or A B  => interp f A ⊔ interp f B
  | .imp A B => interp f A ⇨ interp f B

/-- A context is the meet of its formulas (the empty context is `⊤`). -/
def interpCtx {H : Type} [HeytingAlgebra H] (f : Nat → H) : List Formula → H
  | []     => ⊤
  | A :: Γ => interp f A ⊓ interpCtx f Γ
```

```
/-- **Soundness.** If `Γ ⊢ A` then `⟦Γ⟧ ≤ ⟦A⟧` in every Heyting algebra, for
every interpretation of the atoms. -/
theorem soundness {Γ : List Formula} {A : Formula} (h : Proves Γ A) :
    ∀ {H : Type} [HeytingAlgebra H] (f : Nat → H), interpCtx f Γ ≤ interp f A := by
  induction h with
  | hyp hmem =>
    intro H _ f
    induction hmem with
    | head => exact inf_le_left _ _
    | tail _ _ ih => exact le_trans _ _ _ (inf_le_right _ _) ih
  | topI => intro H _ f; exact le_top _
  | botE _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (bot_le _)
  | andI _ _ ih₁ ih₂ => intro H _ f; exact le_inf _ _ _ (ih₁ f) (ih₂ f)
  | andE1 _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (inf_le_left _ _)
  | andE2 _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (inf_le_right _ _)
  | orI1 _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (le_sup_left _ _)
  | orI2 _ ih => intro H _ f; exact le_trans _ _ _ (ih f) (le_sup_right _ _)
  | orE _ _ _ ih ihA ihB =>
    -- ⟦Γ⟧ ≤ (a ⊔ b) ⊓ ⟦Γ⟧ ≤ (a ⊓ ⟦Γ⟧) ⊔ (b ⊓ ⟦Γ⟧) ≤ c
    intro H _ f
    apply le_trans _ _ _ (le_inf _ _ _ (ih f) (le_refl _))
    apply le_trans _ _ _ (sup_inf_le _ _ _)
    exact sup_le _ _ _ (ihA f) (ihB f)
  | impI _ ih =>
    -- residuation turns `a ⊓ ⟦Γ⟧ ≤ b` into `⟦Γ⟧ ≤ a ⇨ b`
    intro H _ f
    apply (le_himp_iff _ _ _).mpr
    exact le_trans _ _ _ (inf_comm_le _ _) (ih f)
  | impE _ _ ih₁ ih₂ =>
    -- ⟦Γ⟧ ≤ (a ⇨ b) ⊓ a ≤ b
    intro H _ f
    exact le_trans _ _ _ (le_inf _ _ _ (ih₁ f) (ih₂ f)) (himp_inf_le _ _)
```

The statement says that a derivation of $\Gamma \vdash A$ gives $[\![\Gamma]\!] \le [\![A]\!]$ in every Heyting algebra. Each case is one law: Hyp is "a meet is below its parts" (by induction on the position of the formula in the context), ∧I is `le_inf`, the eliminations are `inf_le_left` and `inf_le_right`, ∨E is distributivity followed by `sup_le`, →I is residuation, and →E is modus ponens. The proof uses no axioms at all.

Two instances

The two-element algebra is `Bool`, and its laws are checked by `decide`, which tries every combination of values:

```
/-- The two-element Heyting algebra: the truth values. `a ⇨ b` is `!a || b`. -/
instance : HeytingAlgebra Bool where
  le a b := a = true → b = true
```

```
  min a b := a && b
  max a b := a || b
  himp a b := !a || b
  top := true
  bot := false
  -- Each law is checked on every combination of elements: up to 27 cases.
  le_refl a := by cases a <;> decide
  le_trans a b c := by cases a <;> cases b <;> cases c <;> decide
  le_antisymm a b := by cases a <;> cases b <;> decide
  inf_le_left a b := by cases a <;> cases b <;> decide
  inf_le_right a b := by cases a <;> cases b <;> decide
  le_inf a b c := by cases a <;> cases b <;> cases c <;> decide
  le_sup_left a b := by cases a <;> cases b <;> decide
  le_sup_right a b := by cases a <;> cases b <;> decide
  sup_le a b c := by cases a <;> cases b <;> cases c <;> decide
  bot_le a := by cases a <;> decide
  le_top a := by cases a <;> decide
  le_himp_iff a b c := by cases a <;> cases b <;> cases c <;> decide
```

The three-element chain needs a type of its own:

```
/-- The three-element chain `0 < ½ < 1`: "false", "undecided", "true". -/
inductive Three where
  | zero | half | one
  deriving DecidableEq, Repr

/-- The position of an element in the chain. -/
def Three.rank : Three → Nat
  | .zero => 0
  | .half => 1
  | .one  => 2

/-- In a chain, `a ⇨ b` is `1` when `a ≤ b`, and `b` otherwise. -/
instance : HeytingAlgebra Three where
  le a b := a.rank ≤ b.rank
  min a b := if a.rank ≤ b.rank then a else b
  max a b := if a.rank ≤ b.rank then b else a
  himp a b := if a.rank ≤ b.rank then .one else b
  top := .one
  bot := .zero
  -- Each law is checked on every combination of elements: up to 27 cases.
  le_refl a := by cases a <;> decide
  le_trans a b c := by cases a <;> cases b <;> cases c <;> decide
  le_antisymm a b := by cases a <;> cases b <;> decide
  inf_le_left a b := by cases a <;> cases b <;> decide
  inf_le_right a b := by cases a <;> cases b <;> decide
  le_inf a b c := by cases a <;> cases b <;> cases c <;> decide
  le_sup_left a b := by cases a <;> cases b <;> decide
  le_sup_right a b := by cases a <;> cases b <;> decide
  sup_le a b c := by cases a <;> cases b <;> cases c <;> decide
  bot_le a := by cases a <;> decide
  le_top a := by cases a <;> decide
  le_himp_iff a b c := by cases a <;> cases b <;> cases c <;> decide
```

Each law is checked by `cases` on every argument followed by `decide`: at most 27 combinations. Now the facts computed above:

```
/-- info: Ch05.Three.zero -/
#guard_msgs in
#eval neg Three.half

/-- info: Ch05.Three.one -/
#guard_msgs in
#eval neg (neg Three.half)

/-- Excluded middle fails at `½`: `½ ⊔ ¬½ = ½ ⊔ 0 = ½`, not `1`. -/
example : Three.half ⊔ neg Three.half ≠ ⊤ := by decide
```

```
/-- Double-negation elimination fails at `½`: `¬¬½ = 1`, which is not `≤ ½`. -/
example : ¬ (neg (neg Three.half) ≤ Three.half) := by decide

/-- In `Bool`, both hold for every element. -/
example : ∀ a : Bool, a ⊔ neg a = ⊤ ∧ neg (neg a) = a := by decide
```

And with soundness, a second, algebraic proof that excluded middle is not derivable, independent of Kripke models:

```
/-- A second proof that excluded middle is not derivable, this time
algebraic: interpret the atom `P` as `½`. -/
theorem lem_not_derivable' : ¬ Proves [] (.or P P.neg) := by
  intro h
  have := HeytingAlgebra.soundness h (fun _ => Three.half)
  revert this
  decide
```

Soundness gives `⊤ ≤ ½ ⊔ ¬½` for the interpretation that sends every atom to `½`; `revert this` turns that hypothesis into the goal `⊤ ≤ ... → False`, which `decide` refutes by computing.

The two semantics are closely related: the up-closed sets of worlds of a Kripke model (the "propositions" that respect monotonicity) form a Heyting algebra. The three-element chain comes from the two-world model: the up-sets are $\varnothing$, $\{later\}$ and $\{now, later\}$, which are 0, ½ and 1. Another classic source of Heyting algebras is topology: the open sets of a space, with $U \Rightarrow V$ the interior of $(X \setminus U) \cup V$. In the real line, $U = \mathbb{R} \setminus \{0\}$ has $\neg U = \mathrm{int}\{0\} = \varnothing$ and $\neg\neg U = \mathbb{R} \neq U$: the missing point is undecided.

Negation in a Heyting algebra

The elements with $\neg\neg a = a$ are called **regular**. Every negation is regular (that is `neg_neg_neg`), and one can show that the regular elements form a Boolean algebra. That is Glivenko's theorem in algebraic form: the map $a \mapsto \neg\neg a$ projects intuitionistic truth values onto classical ones, and classical logic is the logic of the regular elements, *inside* intuitionistic logic rather than beyond it. In the three-element chain the regular elements are 0 and 1; the undecided ½ is what the projection forgets.

In Mathlib

Mathlib's `HeytingAlgebra` states residuation the same way, and `Prop` is an instance:

```
/-- Mathlib's class states residuation in the same form as ours. -/
example {α : Type} [_root_.HeytingAlgebra α] (a b c : α) :
    a ≤ b ⇨ c ↔ a ⊓ b ≤ c :=
  le_himp_iff

/-- `Prop` is a Heyting algebra: `≤` is `→`, `⊓` is `∧`, `⊔` is `∨`, `⇨` is
`→`, and the complement `ᶜ` is `¬`. All of these hold by `rfl`. -/
example (p q : Prop) : (p ≤ q) = (p → q) := rfl
example (p q : Prop) : (p ⊓ q) = (p ∧ q) := rfl
example (p q : Prop) : (p ⊔ q) = (p ∨ q) := rfl
example (p q : Prop) : (p ⇨ q) = (p → q) := rfl
example (p : Prop) : pᶜ = ¬p := rfl

/-- The general laws we proved by hand are in Mathlib too. -/
example {α : Type} [_root_.HeytingAlgebra α] (a : α) : a ≤ aᶜᶜ := le_compl_compl
example {α : Type} [_root_.HeytingAlgebra α] (a : α) : aᶜᶜᶜ = aᶜ := compl_compl_compl a
```

Every one of these holds by `rfl`: in Mathlib's instance, `≤` on `Prop` *is* implication, meet *is* `∧`, Heyting implication *is* `→`, and the complement *is* `¬`. Propositions are the truth values of a Heyting algebra, with entailment as the order. Mathlib calls negation the *complement* `aᶜ`, and the laws we proved by hand are library lemmas (`le_compl_compl`, `compl_compl_compl`). Mathlib also makes `Prop` a `BooleanAlgebra`, which needs excluded middle, and `#print axioms` shows it:

```
/-- Excluded middle holds in every Boolean algebra... -/
theorem em_boolean {α : Type} [BooleanAlgebra α] (a : α) : a ⊔ aᶜ = ⊤ :=
  sup_compl_eq_top

/-- ...and Mathlib makes `Prop` a Boolean algebra, using classical logic. -/
theorem em_prop (p : Prop) : p ⊔ pᶜ = ⊤ := em_boolean p

/-- info: 'Ch05.em_prop' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in
#print axioms em_prop
```

5.9 The categorical reading

One more step of abstraction connects this chapter with Part IV. A preorder is a category in which there is at most one arrow between two objects: an arrow $a \to b$ exists exactly when $a \le b$ (Chapter 16 makes this precise). In that reading, meet is a *product*, join is a *coproduct*, $\top$ and $\bot$ are the terminal and initial objects, and residuation

$$c \sqcap a \le b \quad \text{iff} \quad c \le (a \Rightarrow b)$$

has the shape of currying, $\mathrm{Hom}(C \times A, B) \cong \mathrm{Hom}(C, B^A)$. A category with products and this currying property is called **cartesian closed**, and Chapter 19 shows that cartesian closed categories are exactly the models of the simply typed λ-calculus. A Heyting algebra is a cartesian closed category in which all proofs of the same entailment are identified; the general case keeps *proofs* as arrows instead of just "provable or not", which is the Curry–Howard correspondence one level up.

Logic	Categorical structure
intuitionistic	cartesian closed category (with coproducts)
intuitionistic, proofs ignored	Heyting algebra
classical, proofs ignored	Boolean algebra
linear (Chapter 21)	symmetric monoidal closed category
affine	symmetric monoidal closed, with a terminal unit

The last rows preview Part V. Residuation survives when the product $\sqcap$ is replaced by a tensor $\otimes$ that cannot duplicate or discard: the diagonal $a \to a \times a$ is contraction, the map $a \to \top$ is weakening, and dropping them is what turns intuitionistic logic into linear logic, and Rust's moves into an affine type system. Classical logic, on the other hand, does not have a good proof-relevant version of this table: a well-known argument (usually credited to André Joyal) shows that a cartesian closed category with an initial object, in which the canonical arrow $a \to \neg\neg a$ is an isomorphism for every object, collapses to a preorder. Classical proofs need control operators, not just functions.

5.10 Putting it together: the De Morgan law

Let us follow one formula through the chapter: $\neg(A \wedge B) \to \neg A \vee \neg B$.

Classically, it is easy. `not_and_or` splits on `p` with `by_cases`. If `p` holds, `q` must fail, since `p ∧ q` is refuted; if not, `¬p` is the left disjunct. `#print axioms` reports `Classical.choice`.

Constructively, half of it holds. The converse, $\neg A \vee \neg B \to \neg(A \wedge B)$, is `not_and_of_or_not`, with no axioms: given a refutation of one side, refute the conjunction by projecting out that side. The classical direction asks for more: from a refutation of the conjunction, a *decision* about which conjunct is to blame. BHK asks where that decision comes from, and there is no answer.

A countermodel shows it cannot be proved. The Rust search reports that no model with one or two worlds refutes the law, and that the fork does:

```
De Morgan         ¬(P ∧ Q) → ¬P ∨ ¬Q
  3 worlds, 0≤1 0≤2, P@{1} Q@{2}, fails at 0
```

Lean confirms the countermodel by `decide` and turns it into a theorem about the proof system with `kripke_soundness`: `de_morgan_not_derivable`, in the fork section above. The root forces $\neg(P \wedge Q)$ but neither $\neg P$ nor $\neg Q$.

It is not as strong as excluded middle. The two-world model, which refutes excluded middle, validates every instance of this law (it is a chain, so any two futures have a common future). So adding the De Morgan law to intuitionistic logic does not give classical logic: it gives the intermediate logic of weak excluded middle.

Its double negation is constructive. `nn_not_and_or` proves $\neg\neg(\neg(A \wedge B) \to \neg A \vee \neg B)$ without axioms, by borrowing $A \vee \neg A$ inside the double-negation monad, as Glivenko's theorem promises. Classical reasoning is available constructively, as long as the conclusion is negative.

5.11 Common confusions

"**Intuitionists believe excluded middle is false.**" They do not; its negation is refutable, since $\neg\neg(A \vee \neg A)$ is a theorem. Excluded middle is not *assertible* in general, because asserting a disjunction requires knowing which side holds.

"**Proof by contradiction is classical.**" It depends on what is being proved. To prove $\neg A$, assuming A and deriving $\bot$ is just $\to$I, and perfectly constructive. To prove A by assuming $\neg A$ and deriving $\bot$ is RAA, which is classical. Mathematicians use the same phrase for both.

"**Lean is a classical proof assistant.**" Lean's core logic is intuitionistic; classical logic enters through one axiom, `Classical.choice`, which the library uses to prove `Classical.em`. Mathlib uses it freely, and so do some tactics (`grind`, `simp` with classical lemmas, `by_cases`). `#print axioms` tells you which of your results depend on it.

"**A Kripke world is a possible world where every proposition is true or false.**" A Kripke world is a state of knowledge. $w \nVdash A$ does not mean that A is false at w, only that it is not known there; falsity at w is $w \Vdash \neg A$, which constrains every future.

"`Decidable p` **is the same as** `p ∨ ¬p`." Both have two cases, but `Decidable p` is data that a program can branch on, while `p ∨ ¬p` is a proposition whose proofs are erased. Every `Decidable p` gives `p ∨ ¬p`; the converse needs `Classical.choice` and gives up computability.

5.12 Summary

- In intuitionistic logic, truth is provability. The BHK interpretation says what a proof of each connective is, and it is a description of data types: pairs, tagged unions, functions, the unit type and the empty type.
- Excluded middle, double-negation elimination and Peirce's law fail constructively and are equivalent to one another; the De Morgan law $\neg(A \wedge B) \to \neg A \vee \neg B$ fails too, but is weaker.
- In exchange, intuitionistic proofs have the disjunction and existence properties: they carry the information which side and which witness. That is what makes them programs.
- Classical logic embeds into intuitionistic logic by double negation (Glivenko); $\neg\neg(A \vee \neg A)$ is constructive, and $\neg\neg$ behaves like a continuation monad. Classical proofs compute with control operators.
- Negation is $A \to \bot$: a refutation is a function into the empty type. Rust has empty enums, `Infallible` and `!`, and its exhaustiveness checker uses emptiness.
- In Lean, `Classical.em` and `byContradiction` rest on the axiom `Classical.choice`; `Decidable p` is the computable form of excluded middle, needed by `if` and by `decide`; `#print axioms` tells constructive proofs from classical ones.
- Kripke models (worlds of knowledge, monotone forcing, implication quantifying over the future) are sound and complete for intuitionistic logic. A two-world model refutes excluded middle; we proved soundness in Lean and checked the countermodel by `decide`, so excluded middle is not derivable.
- Heyting algebras are lattices with residuated implication. We built the class from scratch, proved soundness of natural deduction in every instance, and refuted excluded middle in the three-element chain. Mathlib's `Prop` is a Heyting algebra with `≤` as implication.
- Categorically, a Heyting algebra is a thin cartesian closed category; dropping the cartesian structure leads to the linear and affine logics of Part V.

5.13 Exercises

1. Find, by hand, a Kripke countermodel to material implication $(P \to Q) \to \neg P \vee Q$, and check each clause of the forcing definition. Then add the formula to the list in `ch05_kripke.rs` and compare with the program's answer. *Hint:* two worlds suffice. P must be undecided at the root, so that $\neg P$ fails there, and $P \to Q$ must hold at the root, so Q must become known wherever P does. (With Q never known, $P \to Q$ fails at the root and the implication holds vacuously.)
2. Prove in Lean, without `Classical`, that `¬¬(p → q) → ¬¬p → ¬¬q` (the double-negation monad applied to a function) and that `¬¬(¬p ∨ ¬¬p)`. Check both with `#print axioms`. *Hint:* for the

first, `intro h hp k` and then `apply h; intro hpq; apply hp; intro hp'; exact k (hpq hp')`. For the second, imitate `nn_em`: commit to the right disjunct first, and when it is challenged, switch to the left.

3. Prove that double-negation elimination implies Peirce's law, constructively: `(∀ p : Prop, ¬¬p → p) → ∀ p q : Prop, ((p → q) → p) → p`. *Hint:* apply the hypothesis to `p`, then `intro k` with `k : ¬p`; to use the assumption `(p → q) → p`, build a function `p → q` from `k` with `absurd`.
4. Add a three-world chain `now ≤ soon ≤ later` to `Kripke.lean`, with `P` known only at `later`. Use `decide` to check that excluded middle fails at `now` and at `soon`, and that `(P → Q) ∨ (Q → P)` holds at every world. *Hint:* copy `Stage` and its instances; the order is "rank of the first ≤ rank of the second", so `le_trans` follows by `decide` after `cases`.
5. In the three-element chain, compute Peirce's law $((a \Rightarrow b) \Rightarrow a) \Rightarrow a$ for $a = ½, b = 0$. Then prove `¬ Proves [] (.imp (.imp (.imp P Q) P) P)` in Lean using `HeytingAlgebra.soundness` and `Three`. *Hint:* $½ \Rightarrow 0 = 0,\ 0 \Rightarrow ½ = 1,\ 1 \Rightarrow ½ = ½$. Interpret atom 0 as `half` and atom 1 as `zero`: `fun n => if n = 0 then Three.half else Three.zero`, and finish as in `lem_not_derivable'`.
6. Extend the Rust search to four worlds and find a frame (an order, without a valuation) on which weak excluded middle $\neg P \vee \neg\neg P$ holds at every world for *every* valuation, but linearity $(P \to Q) \vee (Q \to P)$ fails for some valuation. *Hint:* weak excluded middle holds on every frame in which any two futures of a world have a common future. Try a diamond: a root, two incomparable worlds, and a top above both.